Lab Report: **Regularization Techniques in Image Classification**

Mohsen Alihosseini
12504187
Advanced Programming
Instructor: Prof. Tobias Schaffer

31.05.2025

1 Introduction

In this lab we would check Regularization Techniques including **Dropout**, **L2 Weight** , **Combine Dropout and L2**,**Early Stopping** and **Data Augmentation**.
We would apply these techniques on Dog and Cat images with 20.000 training images (9.985 cat images and 10.015 Dog images)

2 Methodology

As following in each step, we would apply one of the aforementioned techniques on the data to check how these techniques can affect training and validation results. Also, a short summary of each step would be shared to make these steps more clear.

2.1 Software and Hardware Used

- Programming language: e.g. Pythont
- Libraries: tensorflow, sklearn, matplotlib.
- Hardware: T4GPU on colab

2.2 Basic Network

- **Input:** Accepts images of size 150x150 pixels with 3 channels (RGB) after resize images
- **Convolutional Layers:**Three Conv2D layers with increasing filters ($32 \rightarrow 64 \rightarrow 128$) and ReLU activation.
- **Pooling Layers:**after each convolution to downsample the feature maps, reducing spatial dimensions and computation.
- **Flattening:**Converts the 3D feature maps into a 1D vector for the dense layers.

- **Dense (Fully Connected) Layers:** One hidden layer with 128 units and ReLU activation and final output layer with 1 unit and sigmoid activation (for binary classification).

Here is the simple configuration of the Basic Network

```
from tensorflow.keras import models, layers

# Build the model
model_basic = models.Sequential([
    layers.Input(shape=(150, 150, 3)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Compile the model
model_basic.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

2.2.1 Observation on Basic Network

In following the trend of 10 Epoch will be analyzed, this would give us idea that how our model is working and what is the porblem

- **Training:** Accuracy improves dramatically from **57.69%** to **99.14%** over 10 epochs. Loss decreases sharply from **0.6684** to **0.0263**, indicating the model is learning the training data very well.

- **Validation**

  Validation accuracy improves initially (**73.76%** to **83.56%** by epoch 5), but then plateaus and slightly decreases (ending at **81.26%**).

  Validation loss follows a U-shaped curve, reaching its lowest at epoch 5 (0.4136) before increasing significantly to 0.8979.

- **Understanding:** The model is memorizing the training data (99.14% accuracy) but fails to generalize well to validation data (81.26%).

  The growing gap between training and validation metrics is a shows overfitting

Aforementioned trends are shown in Figure 1 for accuracy and Figure 2 for loss.

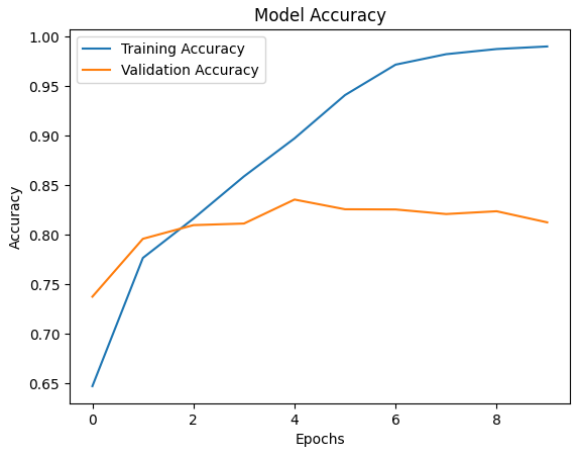


Figure 1: Basic Model Accuracy

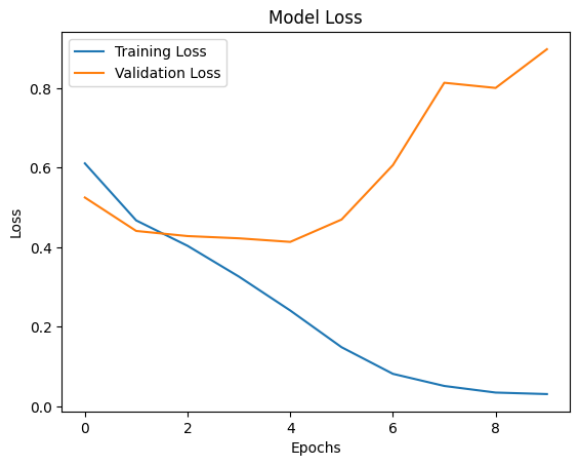


Figure 2: Basic Model Loss

2.3 Dropout

Dropout is a regularization technique in deep learning used to prevent overfitting by randomly deactivating neurons during training. Here in the following Figure 3 and Figure 4 changes show that affect accuracy and loss by applying **0,5** dropout for each layer.

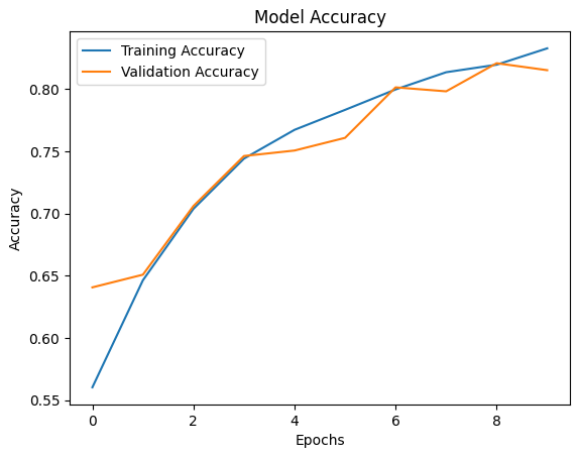


Figure 3: Dropout Accuracy

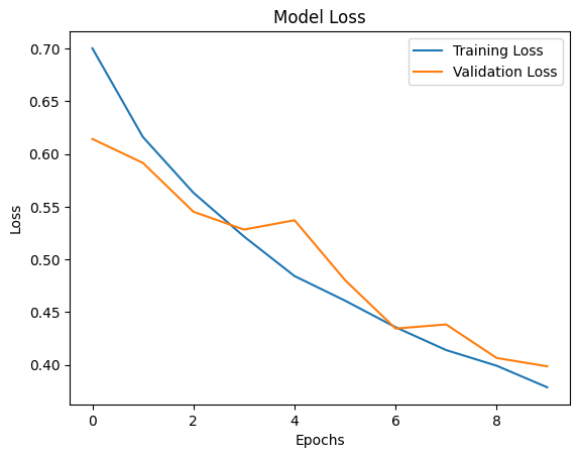


Figure 4: Dropout Loss

2.3.1 Observation:

- **Training Accuracy:**Using dropout show slower growth now 83.4% (Epoch 10) vs. 99.3% (without dropout).
- **Validation Accuracy:**Peaks at 82.1% (Epoch 9) vs. 81.9% (without dropout) and No drastic decline
- **Training & Validation Loss:**Validation loss stabilizes around 0.39–0.53 (no runaway increase). Also Training loss (0.37) and validation loss (0.40) are now much closer, indicating better generalization.

As shown by data, using dropout 0,5 improved loss and prevent overfitting. However using 0,5 dropout may be aggressive as affected accuracy compare to basic model

2.4 L2 Weight Regularization

L2 regularization, also known as ridge regression, is a technique to prevent overfitting by penalizing large weights in a model. This is done by adding a penalty term to the model's loss function, which is proportional to the sum of the squared values of the model's weights. The penalty term encourages the model to have smaller, more evenly distributed weights, reducing the impact of any single feature and promoting better generalization. In following graphs accuracy and loss for L2 Regularization by `kernel_regularizer=tf.keras.regularizers.l2(0.001)` this mean learning rate is λ=0,001

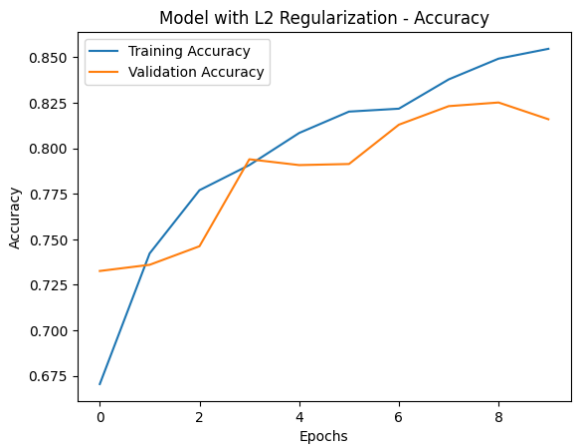


Figure 5: L2 Accuracy

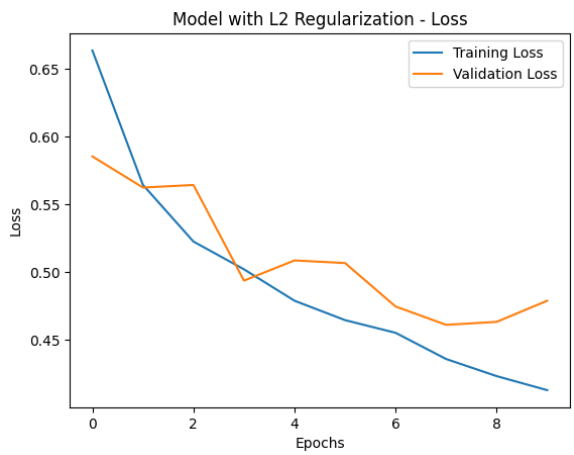


Figure 6: L2 Loss

2.4.1 Observation:

Final training Accuracy is around 80-85% which is lower than basic model. However shows steady increment which may reach higher value by more training epoch. Also final Validation Accuracy is around 81-84% this is around training values. Training and Validation Loss Trend is same and Stable around 0.35-0.45, with no explosion.

2.5 Combine Dropout and L2

In combined format we can benefit from both L2 and Dropout advantages. It mean even in omitted neuron we had a mechanism to control weight from exploding. In following the result presented for this kind of regularization

- **Training Accuracy:** With L2 + Dropout it shows steady growth, reaching 79.9% (Epoch 10) vs. 99.3% (basic model) but Slower progress with no extreme overfitting.
- **Validation Accuracy:**Peaks at 80.4% (Epoch 8) vs. 81.9% (basic model peak).No drastic decline (ends at 76.9% in Epoch 10 vs. basic model's collapse to 80.1% with exploding validation loss).
- **Training & Validation Loss:**Validation Loss Stabilizes between 0.484–0.532 (no runaway increase like the basic model 1.067).Training Loss (0.50) and Val Loss (0.53) are much closer (vs. basic model's 0.02 vs. 1.07), indicating better generalization.

2.5.1 Observation:

This technic could control over fitting as shown there is small gap between training and validation loss and indicate more generalization (Training Loss (0.50) and Val Loss (0.53)). Also accuracy in training and validation had growing trend and small gap where between them. This gap also show better generalization(3-5% gap (train: 79.9%, val: 76.9%) vs. 19% gap in the basic model.)

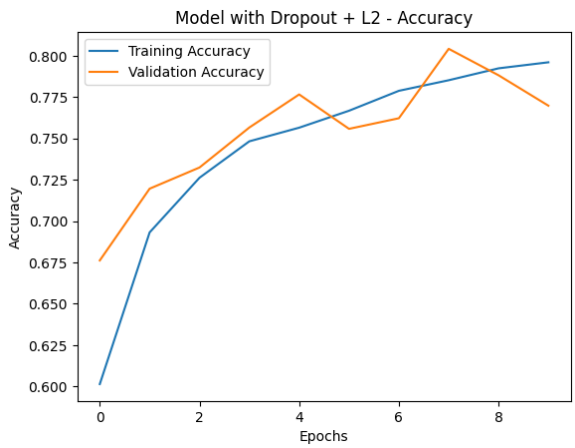


Figure 7: Combined Accuracy

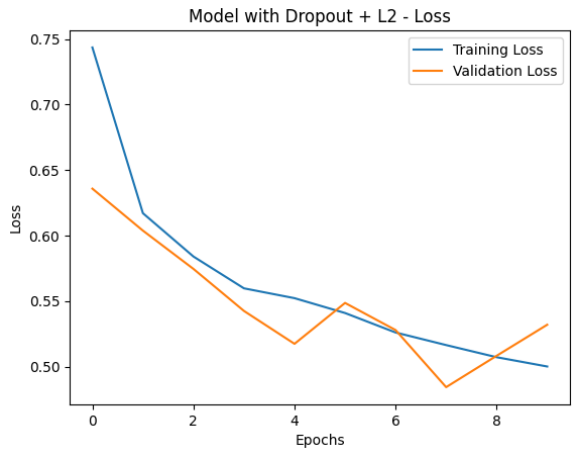


Figure 8: Combined Loss

2.6 Early Stopping

Adding Early Stopping will help prevent overfitting by automatically stopping training when the validation loss stops improving in our case patience set to 3 and number of epoch 50. Number of epoch set to higher value as we are sure if 3 consequent loss value is not improve training will stop: `EarlyStopping(monitor='val_loss', patience=3)`.

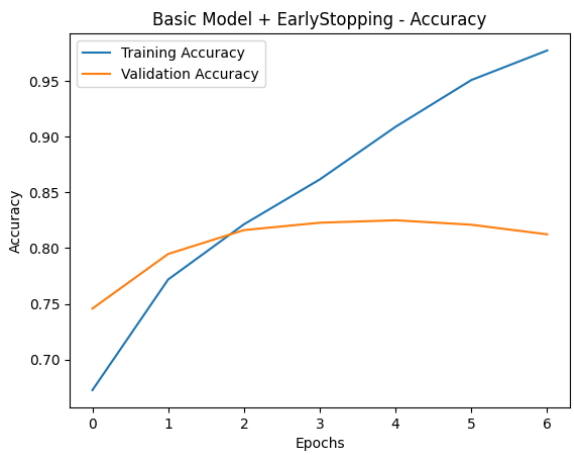


Figure 9: Early Stopping Accuracy

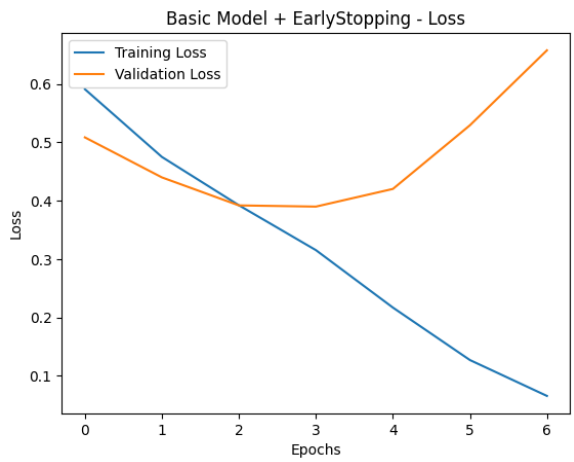


Figure 10: Early Stopping Loss

- **Training Accuracy:**Reaches at 90.9% (Epoch 5) then Prevents overfitting by halting training early. while basic model had better performance in accuracy and reached 99.3% in Epoch 10. This mean training stop earlier to maintain validation loss!

- **Validation Accuracy:**Peaks at 81.9% (Epoch 5), then collapses to 80.1% (Epoch 10). And early stop mechanism start on Epach 5 Best: 82.5% (Epoch 5)

2.6.1 Observation

In basic Model as there is no stopping then always overfit happen by Epoch 10. While using early Stopping Model on Epoch 5 as validation loss not improves (as a sign of overfit) then training stop. Best Epoch: 5 (validation accuracy=82.5%, validation loss=0.4201).

2.7 Data Augmentation

Data Augmentation is a technique used to artificially expand the size and diversity of a training dataset by applying random transformations to the original data.
The data augmentation that we applied here is random transformations to images during training: `rotation_range=20` rotates images up to 20°, `width_shift_range=0.2` and `height_shift_range=0.2` shift images horizontally/vertically by 20%, `horizontal_flip=True` flips images left-right, and `zoom_range=0.2` zooms in/out by up to 20%. Together, they increase dataset diversity, improve model generalization, and reduce overfitting by simulating variations in orientation, position, and scale.
Please note that as this augmentation is completely randomly change training data result may vary every time, as i run for three times and get different result for each run time. Flowing figures are the first try, as shown there is up and down in trend

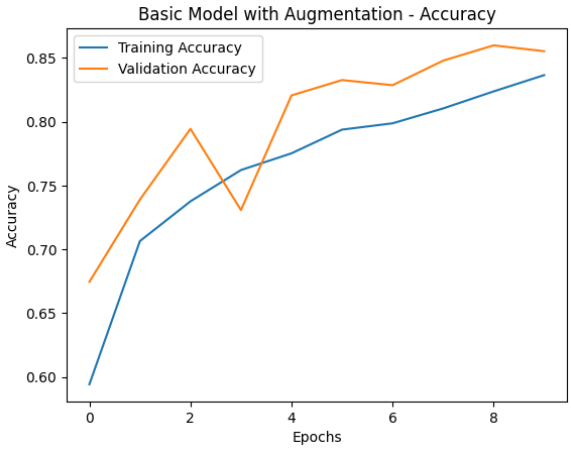


Figure 11: Augmentation Accuracy

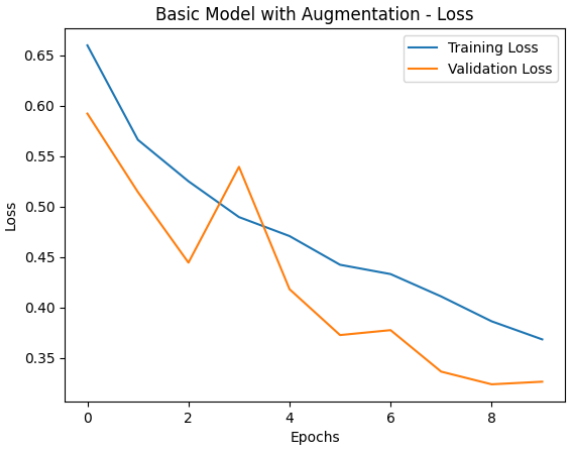


Figure 12: Augmentation Loss

- **Training Accuracy:**Steady growth from 56.9% to 82.3% (slower than basic model's 99.3%).

- **Training Loss:**Smooth descent from 0.68 → 0.39 showing stable learning.

- **Validation Performance**Validation Accuracy peaks at 86.2% (Epoch 10). No decline compare with basic model. Also Validation Loss get lowest at 0.3155 (Epoch 10).

As second try more stable trend. this different result raised because of random augmentation which applied on training images

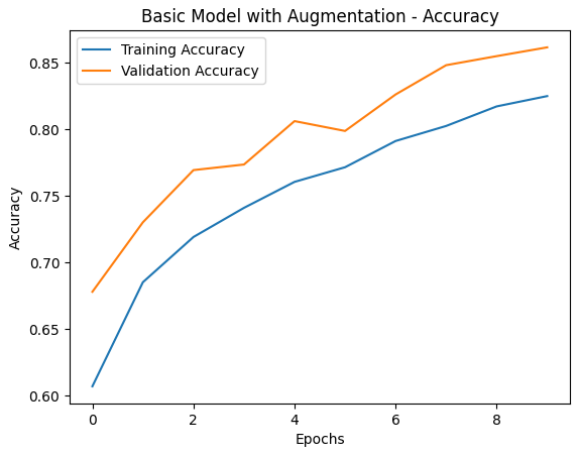


Figure 13: Augmentation Acc Second Try

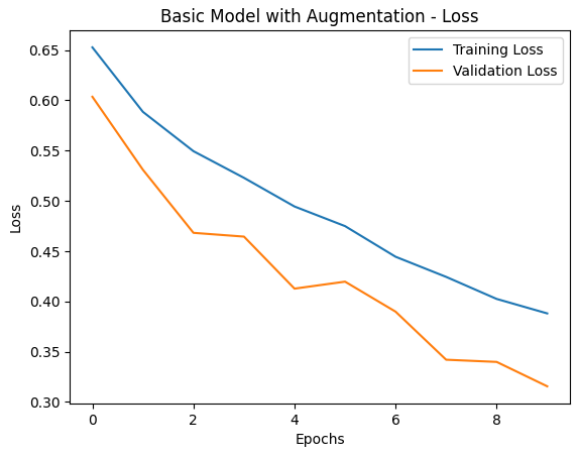


Figure 14: Augmentation Loss Second Try

3 Evaluation and Comparison

The accuracy and loss for all models are presented below. Figure 15 displays the training accuracy and loss. It is evident from this figure that the Basic model's accuracy is comparable to that of the Early Stopping model, while the other models achieve similar accuracy levels. Figure 16 also shows the training loss for all models. From this, it is clear that the Basic model and the Early Stopping model demonstrate better performance compared to the other models.

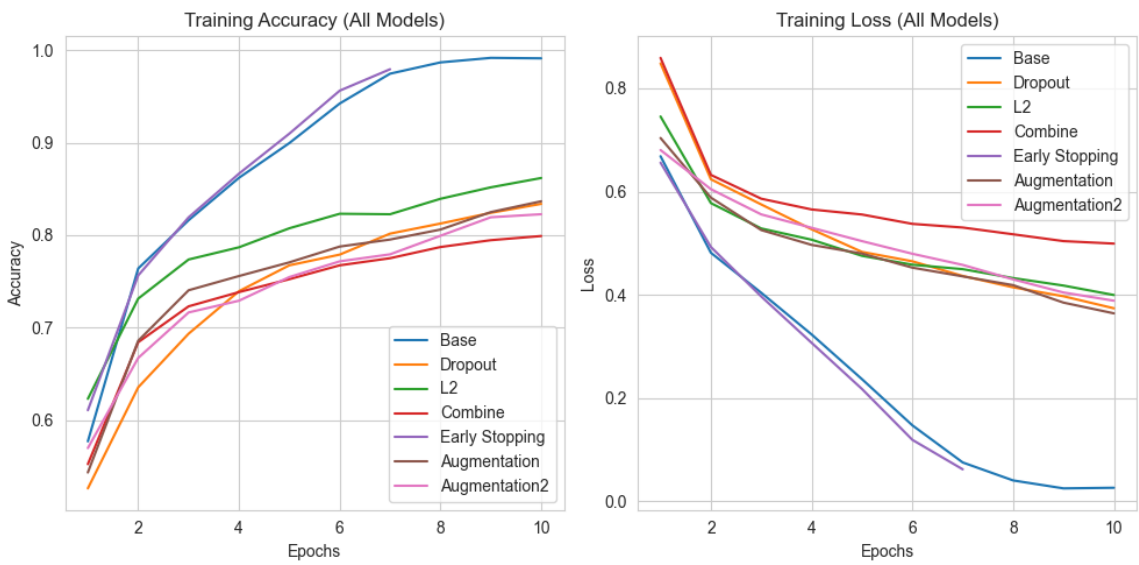


Figure 15: Training Accuracy and Loss for All Tried Models

The validation accuracy and loss for all models are presented in the figure 16. The plot shows that the Base model and Early Stopping model achieve comparable accuracy, while the remaining models perform similarly to one another. The validation loss plot further show better performance for Augmentation. As augmentation help to generalization by changing image randomly,in contrast Base model and early stpping model which are somehow like each other exploded in loss after 5th Epoch. this is a great sign for overfitting. Augmentation have better loss too.

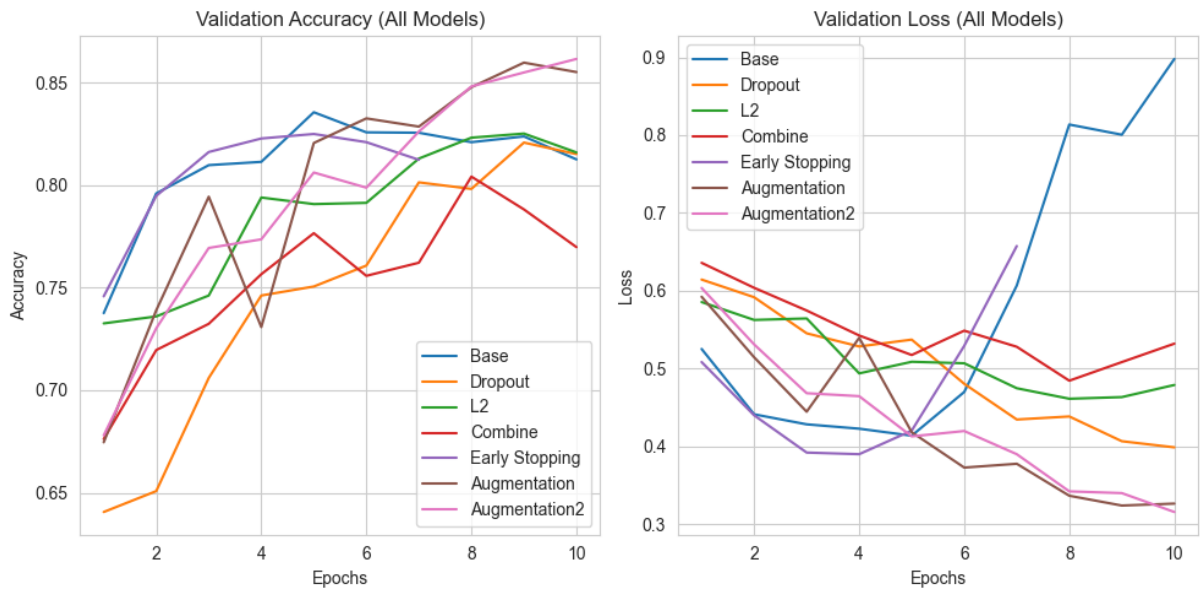


Figure 16: Validation Accuracy and Loss for All Tried Models

There is an important golden pint here, Training accuracy (Table 1) and loss (Table 2) shows better performance in validation than training, it mean unseen image predicted better than seen data. This is a paradigm for a model!

Table 1: Model Accuracy for Training and validation Comparison

Model	**Train Acc.**	**Val. Acc.**	**Acc. Diff.**
Augmentation	0,84	0,86	-1,85%
Augmentation2	0,82	0,86	-3,89%

Table 2: Model Loss for Training and validation Comparison

Model	**Train Loss**	**Val. Loss**	**Loss Diff.**
Augmentation	0,36	0,33	+3,78%
Augmentation2	0,39	0,32	+7,31%

3.1 Summary

In following Summary for the action will peresent

3.1.1 Table

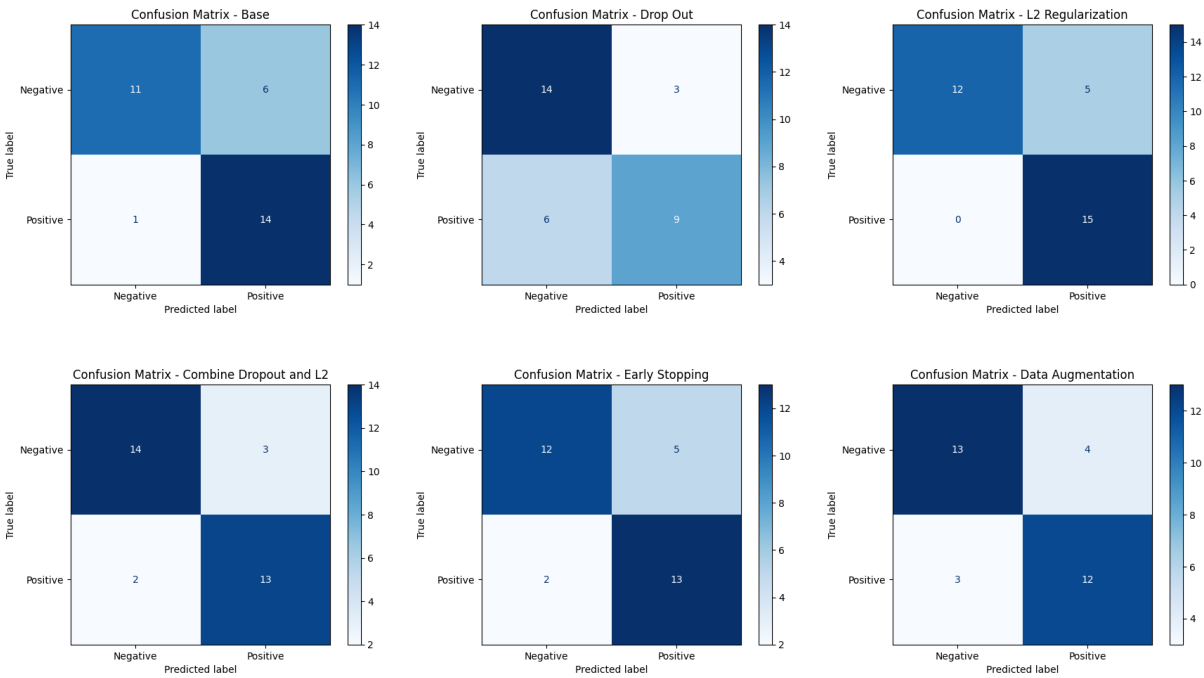
Here is the summary table(Table 3) to indicate validation accuracy for all model and number of epoch to converge by estimation

Table 3: Regularization Techniques Comparison

Technique	**Augm.**	**Val. Acc.**	**Epochs**
Basic Model	No	0.8126	10 (overfit@5)
Dropout	No	0.8152	10 (15-20)
L2	No	0.8160	10 (15-20)
Combined	No	0.7698	10 (15-20)
Early Stop	No	0.6574	7 (stopped)
Augmentation	Yes	0.8552	10 (15-20)
Augmentation2	Yes	0.8616	10 (15-20)

3.1.2 Confusion Matrix

In following confusion matrix for each case will present



3.1.3 Misclassified Validation Images

In Figure 17 a few misclassified validation images along with their predicted class and confidence score (probability) shown. Also their true label indicated

Figure 17: Misclassified Validation Images

3.2 Code Repository

The full source code for this project is available on GitHub at:

`https://github.com/mohsen-alihosseini/AP_Lab7`

This repository includes two notebook, `1_Notebook Dogs vs Cats...` is the notebook that all data for analysis are taken from, but `2_Notebook Dogs vs Cats...` is which that all code runs again and confusion matrix calculated based on its result.

3.3 Conclusion

The Augmentation2 model emerged as the best-performing setup, achieving the highest validation accuracy (86.2%) and lowest validation loss (0.316), shows superior generalization compared to other configurations. Augmentation in validation had even better performance than training! Regularization techniques mitigated overfitting by reducing the gap between training and validation metrics. As show in Basic model as training procedures continues (after Epoch 5) the validation loss increases, this shows network is overfiting which lead to bad performance in unseen images ehich lead to less generalization. However, data augmentation proved most impaction: models like Augmentation2 maintained modest training accuracy (82%) while maximizing validation performance, this is perfect for generalization.